

Темплейти. Необходими
функции в темплейтен клас/функция.

Темплейтни специализации. SFINAE.

Concepts. Вариантични темплейти и `std::tuple`

Семинар 9
Серхан

- При идентични функции и разлики само в типа - използваме темплейти

```
template <typename T>
```

```
T mymax(const T&a, const T&b)
```

```
{ return a > b ? a : b; }
```

- Компиляторът генерира версията за нужния тип при употреба:

```
mymax(3, 14, 2.7); → генерира double mymax
```

```
mymax('a', 'c'); → генерира char mymax
```

Полиморфизъм по време на компилация - темплейтите се разрешават преди програмата да е стартирала

1. Темплейтна функция

```
template <typename T>
```

```
T add(const T&a, const T&b)
```

```
{ return a + b; }
```

- `typename` и `class` са еквивалентни

- предпочитаме `typename`

```
add(1.5, 2.5) // T → double
```

```
add<int>(1, 2); // T = int (изрично)
```

```
add(1, 2.5) → не може, не са от един и същ тип
```

Не-типов параметър - може да е стойност, не само тип

```
template <typename T, int SIZE>
```

```
class FixedArray {
```

```
    T data[SIZE];
```

```
    ...
```

```
};
```

```
FixedArray<int, 5> arr1; → масив от 5 int-a
```

```
FixedArray<double, 3> arr2; → масив от 3 double-a
```

2. Темплейтен клас - контейнер / структура може да работи с различен тип

• Деклариране:

```
template <typename T>
class Stack {
    T* data;
    int top;
    int capacity;
public:
    explicit Stack(int cap=16);
    ~Stack();
    void push(const T& value);
    T pop();
    const T& peek() const;
    bool isEmpty() const;
    int size() const;
};
```

• Дефиниране:

```
template <typename T>
Stack<T>::Stack(int cap): top(-1),
                        capacity(cap)
{
    data = new T[capacity];
}
```

```
template <typename T>
Stack<T>::~~Stack()
{
    delete[] data;
}
```

- Всеки метод изисква:

1. template <typename T>
2. Stack<T>::

Дефиниция в хедъра !

stack.hpp - цялото съдържание е тук
- първо декларации,
после дефиниции

• h - хедър, само с декларации

•hpp - хедър с декларации и дефиниции на темплейтен клас

SFINAE - при инстанциране, ако не се намери функция, версията се изключва и компилаторът търси следваща; ако никоя функция (версия) не е подходяща → грешка

std::enable_if : при Condition=false, няма type и заместването се проваля
при Condition=true, type съществува

Concepts - именувано ограничение върху темплейтен параметър.

Вариантични темплейти - приемат произволен брой параметри от произволни типове

template <typename ... Args> → parameter pack
void f(Args ... args) → argument pack
{ ... ;}

std::tuple - хетерогенен контейнер, съдържа 3 броя елемента от различни типове

• създаване: std::tuple<int, std::string, bool> t1(1, "Иван", true);
↳ изрично казваме типовете

auto t2 = std::make_tuple(42, 3.14, "world");
↳ определя се е tuple ↳ типовете се дедуцират

достъп: std::get<0>(t1) → взима елемента на позиция 0 в tuple t1 → числото 1
std::get<2>(t2) → взима елемент 2 в tuple t2 → "world"
std::get<int>(t2) → казваме му какъв тип да търси и да върне → 42

разпакуване: std::tie(age, name, grade) = t;

размер: tuple_size<T>::value → връща стойността на размера
tuple_element<1, T>::type → връща типа на втория елемент в tuple
↓
индекса

std::forward<T>(arg) - запазва оригиналната категория

Дизайн патърн Итератор

- Итератор - обект, който позволява обхождане на контейнер, без да знае вътрешността. Отделя логиката на обхождане от логиката на контейнера.
- За да работи `for(auto & el: container)` и `std::sort()` трябва да имаме:
 - * `operator *` ^{достъп до ел.}
 - * `operator +`, `operator -`, `operator []`
 - * `operator ++` ^{преместване} `напред` * `operator <` - сравнение
 - * `operator !=`
 ^{проверка за край}